THE UNIVERSITY OF WARWICK

Paper Details

Paper Code: CS2600

Paper Title: Algorithms

Exam Period: April 2024

Exam Rubric

Time allowed: 2 hours

Exam Type: Standard Examination

Instructions

Answer **ONE** question from Section A and **TWO** questions from Section B.

Indicate clearly which three questions you have answered. Among the questions attempted, only the first one in Section A and the first two in Section B (in their order of appearance in the answer book) will be marked.

Read carefully the instructions on the answer booklet and make sure that the particulars required are entered on each answer booklet.

Calculators are not allowed.
The exam question paper MUST NOT be removed from the examination venue.

Section A Answer **ONE** question from this section

1. Recall that a graph $G = (V, E)$ is bipartite if we can assign each node to one of two colours (red and blue), so that no edge connects two nodes of the same colour.

   (a) Give a graph with four nodes that is bipartite. [4]
   Give a different graph with four nodes that is **not** bipartite.

   (b) Describe an algorithm that, given a graph G with n nodes and m edges, will determine whether G is bipartite or not. For this question, you do **not** need to give a proof of correctness. What is the running time of your algorithm? [8]

   The k-colouring problem is, given a graph $G = (V, E)$, to assign one of k colours to each node, so that no edge connects two nodes of the same colour.

   (c) Show that the k-colouring problem is in the complexity class NP. For this question, you do **not** need to show NP-hardness. [8]

   The k-edge colouring problem is, given a graph $G = (V, E)$, to assign one of k colours to each *edge*, so that no node has two incident edges that are the same colour.

   (d) Suppose you had an algorithm that solves the k-colouring problem. Show how you could use it to solve the k-edge colouring problem, and justify your answer. [10]

Total for Question 1: 30

2. Recall that the Strassen algorithm for matrix multiplication computes the product of two $n \times n$ matrices with 7 multiplications of $\frac{n}{2} \times \frac{n}{2}$ matrices.

   Alfie claims to have a better algorithm which beats this, by finding the product of two $n \times n$ matrices using 49 multiplications of matrices that are $\frac{n}{4} \times \frac{n}{4}$, plus a constant number of additions and subtractions of the resulting matrices. Alfie attempts to write the recurrence for the running time of the algorithm, and comes up with $T(n) = T(49 * n/4) + O(n)$.

   (a) Alfie's attempt is incorrect. Write a correct recurrence for this matrix multiplication algorithm. [4]

   (b) Solve the recurrence to give the running time for Alfie's algorithm. [8]

   (c) Is this algorithm asymptotically better than Strassen's algorithm for matrix multiplication? No calculator is required to answer this question. [4]

   (d) Alicia has an algorithm to multiple two n-bit integers together. She has written the recurrence $T(n) = 3T(n/3) + n$ and $T(1) = 1$. Alicia thinks that $T(n) = O(n)$, since if we substitute this in her recurrence we have $3T(n/3)+n = 3O(n/3)+O(n) = O(n)$. Give the correct solution to this recurrence using the recursion tree method, and explain why your answer is different to Alicia's. [7]

   Consider the following pseudocode description of a recursive algorithm:

```
Solve(n, (A, B, C)):
   If n=0, do nothing
   Else
      Call Solve(n-1, (A, C, B))
      Output (A, B)
      Call Solve(n-1, (C, B, A))
```

   (e) Write a recurrence for the number of times that the `Output` statement is executed when `Solve(n, (A, B, C))` is called, then find a closed-form expression for this as a function of n. Prove your answer is correct using induction. [7]

Total for Question 2: 30

Section B Answer **TWO** questions

3. Given a sequence of values, a *subsequence* is a sequence that can obtained by deleting some items from the original sequence. For instance, given the sequence of digits

$$[3, \quad 1, \quad 4, \quad 1, \quad 5, \quad 9, \quad 2, \quad 6, \quad 5, \quad 4]$$

one possible subsequence is

$$[4, \quad 5, \quad 6, \quad 5, \quad 4]$$

A *longest increasing subsequence* of a sequence of values A is a sequence B such that (i) B is a subsequence of A (ii) B is a (strictly) increasing sequence: $B[i] < B[j]$ for each $1 \leq i < j \leq \text{length}(B)$ and (iii) there are no longer subsequences of A satisfying (ii). Note that there may be more than one longest increasing subsequence.

(a) Find a longest increasing subsequence of the sequence [5]

$$[19, \quad 2, \quad 4, \quad 1, \quad 2, \quad 9, \quad 3, \quad 15, \quad 5, \quad 20, \quad 7, \quad 14, \quad 8, \quad 13, \quad 11, \quad 12, \quad 3]$$

Briefly justify your answer.

(b) What is the maximum number of subsequences of a sequence of length n? Hence, what is the running time of a brute-force algorithm that considers every possible subsequence, tests whether it is an increasing sequence, and reports the longest of these? [5]

(c) Give as fast an algorithm as you can to find the *length* of the longest increasing subsequence of a sequence. Assume that the sequence is given to you as an array of n positive integers. What is the running time of your algorithm as a function of the input sequence length n? *Hint:* dynamic programming is a good approach. [15]

A longest *decreasing* subsequence B of sequence A is defined identically to longest increasing subsequence, except that B should be a decreasing sequence, not increasing. That is, $B[i] > B[j]$ for each $1 \leq i < j \leq \text{length}(B)$

(d) How would you use an algorithm for longest increasing subsequence length to find the length of the longest decreasing subsequence? Assume that the given algorithm operates on sequences of n positive integers. Describe a reduction that will transform an instance of the longest decreasing subsequence problem of length n into an instance of the longest increasing subsequence problem. Use this to give an upper bound on the time complexity of the longest decreasing subsequence problem. [10]

Total for Question 3: 35

4. This question is on the tractability of algorithmic problems.

   (a) Describe what is meant by the operator $\leq_P$ in terms of relationships between algorithmic problems. [5]

   (b) Prove rigorously that the relation described by the $\leq_P$ operator is transitive. [5]

   Recall that the SAT algorithmic problem is to determine, given a propositional logic formula, whether that formula can be satisfied by any assignment of variables.

   The k-SAT algorithmic problem (also called k-CNFSAT) is similar to SAT, with the additional constraints that the formula is provided in conjunctive normal form, and each clause has at most k literals for some positive integer $k \geq 1$. So a small instance of 3-SAT may be

   $$(x \vee \neg y \vee z) \wedge (\neg x \vee y \vee \neg z) \wedge (x \vee y \vee z) \wedge (\neg x \vee x \vee \neg z).$$

   (c) Is the above instance satisfiable? Justify your answer. [5]

   (d) For which values of k is k-SAT tractable? Justify your answer briefly. [10]

   Suppose that, instead of checking whether a formula (in CNF, with at most k literals per clause) has a satisfying assignment, we want to check whether such a formula has *at least two* satisfying assignments. Call this algorithmic problem k-DOUBLESAT.

   (e) Prove that k-DOUBLESAT is complete for the complexity class NP, for all $k \geq 3$. [10]

Total for Question 4: 35

5. This question is on variants of flow networks.

   Recall that a *flow network* is a integer-weighted finite directed graph with distinguished *source* and *sink* nodes.

   (a) We can conjoin two flow networks A and B (made of disjoint sets of nodes) to form $A \circ B$ by adding an edge with infinite capacity from the sink of A to the source of B. Give and justify an upper bound for the maximum flow in $A \circ B$. [5]

   (b) Consider a relaxation of flows in which the flow along an edge is allowed to be negative (travel in the "other direction" of the edge), but the magnitude of the flow along that edge is still bounded by the weight of the edge. Does this relaxation ever increase the maximum flow? Justify your answer. [5]

   Recall that a *circulation network* is a variant of a flow network without dedicated source and sink nodes, and where each node has assigned to it a net demand, which may be positive or negative. A *circulation* is an assignment of nonnegative integer flow to each edge such that the net in-flow at every node is equal to its demand. Let CIRCULATION be the algorithmic problem of determining whether a circulation exists for a circulation network.

   (c) Give and justify a polynomial reduction from the max-flow problem in flow networks to CIRCULATION. *Hint: consider the running time of algorithms for max-flow.* [5]

   (d) Give and justify a polynomial reduction from CIRCULATION to the max-flow problem in flow networks. [10]

   (e) Aliya introduces a new algorithmic problem, EDGY-CIRCULATION, which asks whether, given a circulation network, a circulation exists that assigns a flow of exactly 1 to at least one edge. [10]

   She claims that EDGY-CIRCULATION can be used to check for the presence of (directed) cycles in a directed graph $G = (V, E)$ by the following process:

   1. Construct a circulation network from G, such that the demand of every vertex is 0 and the capacity of every edge is 5.
   2. Return Yes if and only if EDGY-CIRCULATION returns Yes.

   Prove or disprove Aliya's claim.

Total for Question 5: 35

End